

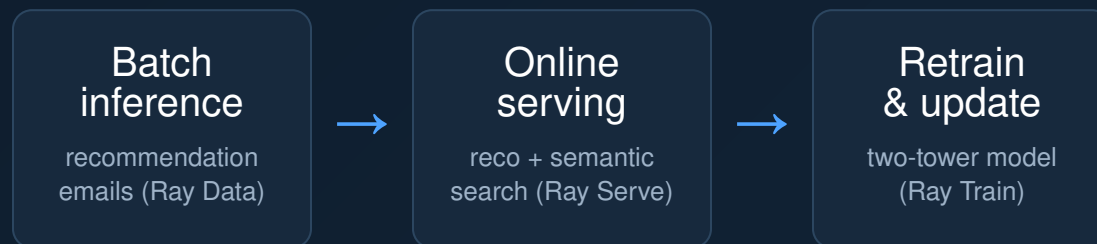
RAY • ANYSCALE • PART 2

Recommendation, Search & Retraining

Continuing the eCommerce catalog: batch & online inference with Ray Data and Ray Serve — semantic search, a two-tower recommender, and periodic model retraining with Ray Train.

Where we are — and what's next

Same eCommerce scenario & data (product catalog + description embeddings from Part 1). Now we put a model to work and keep it fresh.



Three production use cases, one cluster, one programming model — tasks, actors, and `map_batches`.

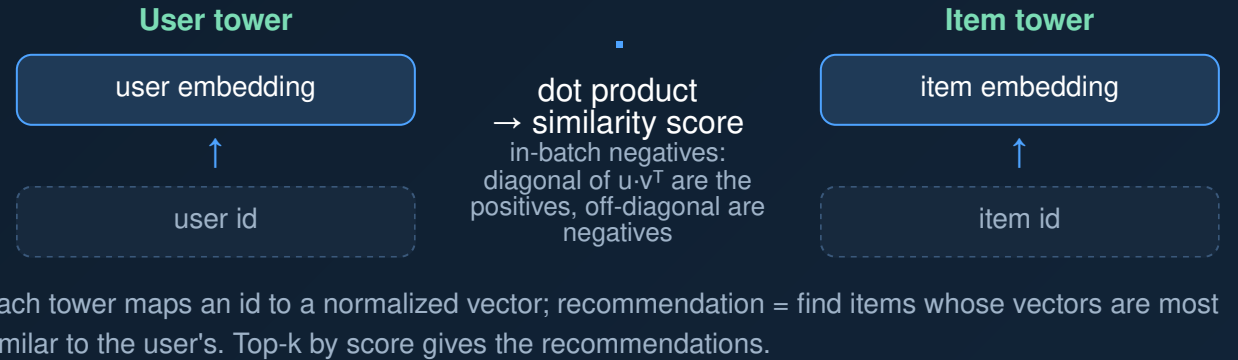
The model: a two-tower recommender

```
class TwoTower(nn.Module):
    def __init__(self, n_users, n_items, dim=64):
        self.user_emb = nn.Embedding(n_users, dim)
        self.item_emb = nn.Embedding(n_items, dim)

    def encode_users(self, uid):
        u = self.user_emb(uid)
        return F.normalize(u, dim=-1)

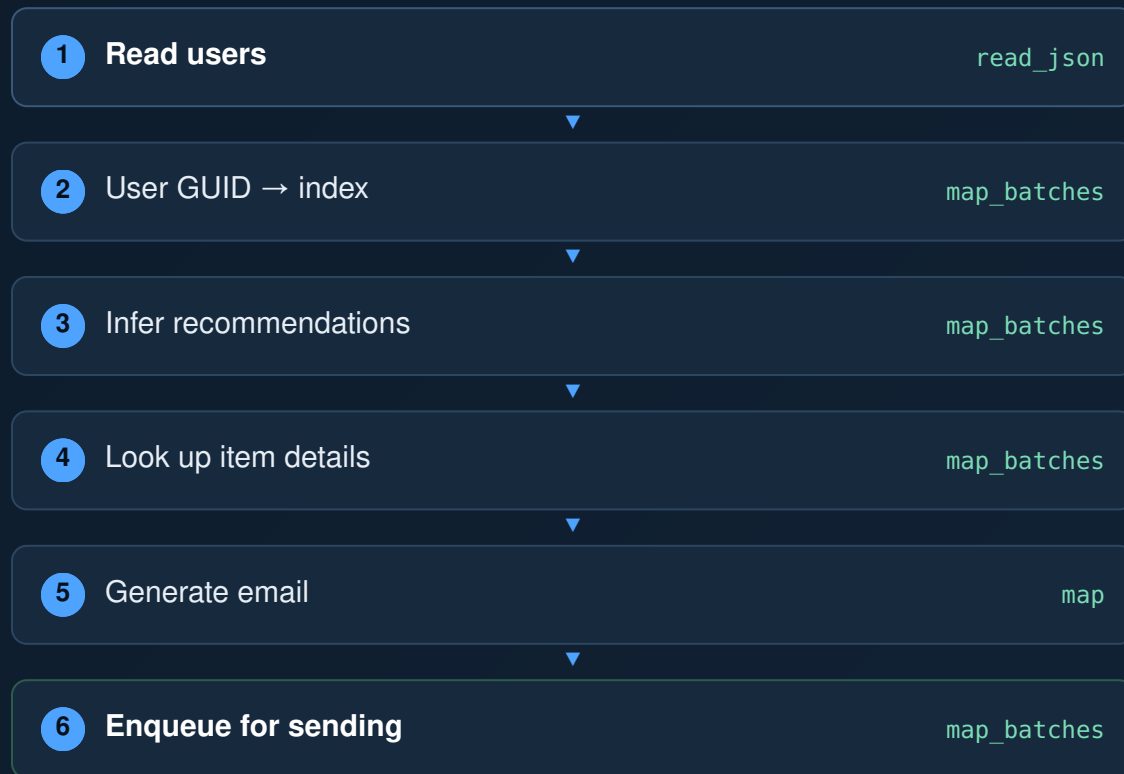
    def encode_items(self, iid):
        v = self.item_emb(iid)
        return F.normalize(v, dim=-1)

    def forward(self, uid, pos_iid):
        u = self.encode_users(uid)      # [B,D]
        v = self.encode_items(pos_iid)  # [B,D]
        return u @ v.t()                # [B,B]
```



Batch inference: recommendation emails

Task: for every user, find products to recommend, write a personalized email, and enqueue it — as one Ray Data pipeline of composable steps.



Each step is an independent, testable transform. Stateless steps use plain functions; steps that hold state (the model, a queue client) use a class that Ray runs as an actor.

Why a pipeline? Datasets are lazy & streaming — build and test step-by-step on a small sample, then run the whole thing at scale without changing the code.

Real systems read users & products from databases. We front that with a **named Ray actor** as a database facade — looked up by name anywhere in the pipeline.

Reading data — and a real gotcha

Ray Data reads many formats, but defaults can surprise you. Troubleshoot by comparing against PyArrow / Pandas.

```
# JSON lines read returns NO rows... no error, no data!
ds = ray.data.read_json('.../users.ndjson', lines=True)
ds.materialize()                                # -> 0 rows

# PyArrow & Pandas read it fine -> data & libs are OK
# Cause: Ray Data filters to *.json by default, skipping .ndjson
ds = ray.data.read_json('.../users.ndjson', lines=True,
                        file_extensions=['.ndjson']) # fixed
```

`materialize()` caches the dataset in the object store — ideal for inspecting and iterating on small data while developing. Lesson: when results look wrong, isolate Ray Data from your data and other libraries.

A named actor as a service facade

Define & name the actor

```
@ray.remote
class DatabaseFacade:
    def __init__(self, users, products):
        self.users = pd.read_json(users, lines=True)
        self.products = pd.read_parquet(products)
    def users_for_ids(self, ids):
        return self.users[self.users['id'].isin(ids)]
    def products_for_indices(self, idxs):
        return self.products.iloc[idxs]

# create once, register a name
DatabaseFacade.options(name="database") \
    .remote(users_path, products_path)
```

Look it up anywhere — even in a stateless step

```
def get_user_indices(batch):
    db = ray.get_actor("database")      # by name
    ref = db.users_for_ids.remote(batch['id'])
    batch['user_index'] = ray.get(ref).index.values
    return batch

ds.map_batches(get_user_indices)
```

The actor holds state (loaded tables); pipeline functions stay stateless and just call it. Remote calls return `ObjectRef`s; `ray.get` fetches the value.

The model step: a stateful `map_batches` actor

The recommender loads the model once in `__init__`; `__call__` scores a batch of users and attaches top-k items. No `@ray.remote` needed — Ray makes it an actor.

```
class Recommend:
    def __init__(self, model_location, num_recommendations, num_users, num_items):
        self.model = TwoTower(num_users, num_items)
        self.model.load_state_dict(torch.load(model_location, weights_only=True))
        self.model.eval()
        self.num_recommendations = num_recommendations

    def __call__(self, batch, column):
        # column = "user_index"
        batch['recommended_items'] = self.recommend_for_users(batch[column])
        return batch

ds.map_batches(get_user_indices) \
    .map_batches(Recommend, fn_constructor_args=[model_path, 3, 1000, 1000],
                fn_args=['user_index'])
```

Finish the pipeline — emails, queue & blocks

```
# per-row email text -> use map()
def generate_email_for_user(user):
    user['reco_email'] = render(user)
    return user

# stateful queue adapter -> map_batches (an actor)
class EmailSendingQueue:
    def __init__(self, service_info):
        self.queue = {'service': service_info, 'out': []}
    def __call__(self, batch):
        self.queue['out'] += list(batch['reco_email'])
        batch['queued'] = [True]*len(batch['reco_email'])
        return batch

# the full pipeline, end to end:
ds.map_batches(get_user_indices) \
    .map_batches(Recommend, fn_args=['user_index'],
                 fn_constructor_args=[path, 3, 1000, 1000]) \
    .map_batches(get_recommended_items_details) \
    .map(generate_email_for_user) \
    .map_batches(EmailSendingQueue, batch_size=32,
                 fn_constructor_args=['sender:bulk']) \
    .count()
```

Gotcha: blocks vs. batches

Asking for a small `take_batch(2)` still processed all 1000 records — because the tiny dataset is a **single block**, and Ray Data must process a whole block at a time.

`repartition(n)` creates more blocks, so a small take touches fewer records. This matters a lot when each inference is slow/expensive (e.g. an LLM) during iterative development.

map = per row | **map_batches** = per batch + reusable actor state. Side effects (sending mail) become a column marking what was queued.

Online serving with Ray Serve

Start simple: one **deployment** handling request I/O (ingress) and recommendation logic. Decorate a class, bind it, run it.

```
@serve.deployment
class IngressAndRecommender:
    def __init__(self, base_model_path, num_recos, num_users, num_products):
        self.db = ray.get_actor("database")
        self.recommender = Recommend(base_model_path, num_recos, num_users, num_products)

    async def __call__(self, request: Request):          # HTTP entrypoint
        user = json.loads(await request.json())
        return self.recommend([user['id']])[['item_id', 'name', 'desc', 'price']].to_json()

handle = serve.run(IngressAndRecommender.bind(base_model_path, 3, 1000, 1000))
requests.post("http://localhost:8000/", json='{"id": "7034dd99-..."}').json()
```

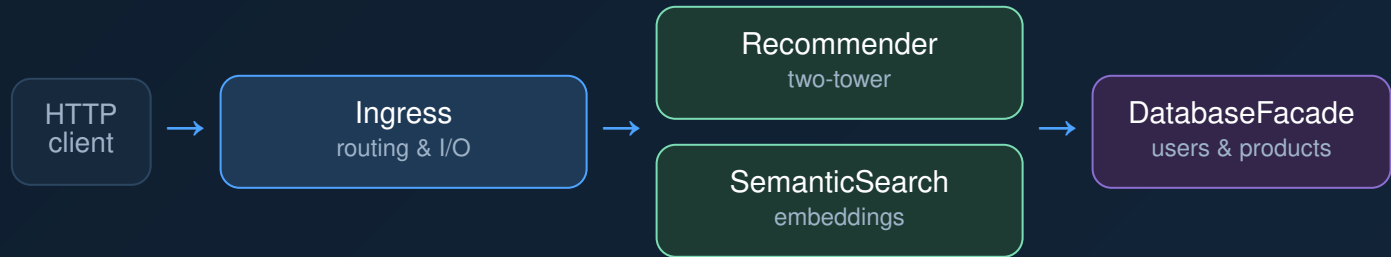
A Serve deployment is a managed, scalable group of replicas. Next we split responsibilities into separate, composable deployments.

Composing deployments (separation of concerns)

```
@serve.deployment
class Ingress:
    def __init__(self, recommender, search):
        self.recommender = recommender
        self.search = search
    async def __call__(self, request):
        u = json.loads(await request.json())
        # await (not ray.get) a deployment handle
        recos = await self.recommender.recommend \
            .remote([u['id']])
        ...

# bind in dependency order
db = DatabaseFacade.bind(users, products)
rec = Recommender.bind(model, 3, 1000, 1000, db)
ing = Ingress.bind(rec, search)
serve.run(ing)
```

DEPLOYMENT GRAPH



Each component scales, upgrades, and recovers independently. With handles you **await** results (async web pattern) rather than `ray.get`. Making the DB a deployment lets Serve restart and autoscale it too.

Semantic search as a deployment

Reuse the Part-1 description embeddings: encode the query, rank by vector similarity, then rerank by name string-distance.

```
@serve.deployment
class SemanticSearch:
    async def __init__(self, model, db):
        self.model = SentenceTransformer(model)
        self.db = db
        all_p = await self.db.all_products.remote()
        self.embs = all_p['desc_emb']

    async def search(self, query, matches):
        sims = self.model.similarity(
            self.model.encode(query), self.embs)
        top = sims.flatten().topk(50).indices
        products = await \
            self.db.products_for_indices.remote(top)
        # rerank by name similarity
        best = torch.tensor([lcsstr(query, n)
                             for n in products['name']]).topk(matches).indices
        return products.iloc[best]
```

Ingress routes on the request

```
async def __call__(self, request):
    user = json.loads(await request.json())
    result = {"recommendations":
              await self.recommender.recommend.remote(
                  [user['id']]})
    if "query" in user: # optional search
        result["search_results"] = \
            await self.search.search.remote(
                user['query'], 5)
    return json.dumps(result)
```

One endpoint returns recommendations always, and adds search results when the request includes a `query`.

Production deploy: config & CLI

Generate a config from code, then tune replicas, resources, and autoscaling per deployment — no code changes.

```
# scaffold a config from the bound app
serve build search_and_recommend:bound_ingress \
  -o serve_config.yaml

# deploy / update from the CLI
serve deploy serve_updated.yaml
serve shutdown -y
```

```
runtime_env:
  working_dir: "https://.../recommend.zip"
deployments:
- name: DatabaseFacade
  num_replicas: 4
- name: Recommender
  num_replicas: 2
  ray_actor_options: {num_cpus: 1, num_gpus: 0.5}
- name: SemanticSearch
  autoscaling_config: {min_replicas: 2, max_replicas: 4}
- name: Ingress
  autoscaling_config: {min_replicas: 2, max_replicas: 4}
```

Per-deployment control

- **num_replicas** — fixed parallelism for a component.
- **autoscaling_config** — min/max replicas that scale with load.
- **ray_actor_options** — CPUs and **fractional GPUs** (e.g. `num_gpus: 0.5` packs two replicas per GPU).
- Health-check & graceful-shutdown timeouts per deployment.

Watch out for runtime details: working dir, imported modules, env vars, and name scoping — Python wasn't designed for a distributed runtime, so be explicit.

Retraining: PyTorch → Ray Train (DDP)

Keep the model fresh with recent interactions. Porting a plain training loop to distributed data-parallel is a small change.

The port

```
def train_loop_ray(config):
    model = TwoTower(config['num_users'],
                      config['num_items'], config['dim'])
    # device detection, data movement, DDP wrap:
    model = ray.train.torch.prepare_model(model)
    opt = torch.optim.Adam(model.parameters(), lr=1e-3)
    for step in range(200):
        ...                # same training math
```

Orchestrate with TorchTrainer

```
trainer = TorchTrainer(
    train_loop_ray,
    scaling_config=ScalingConfig(
        num_workers=2, use_gpu=True),
    run_config=RunConfig(
        storage_path='/mnt/cluster_storage'),
    train_loop_config={'num_users': 1000,
                       'num_items': 1000, 'dim': 64},
)
result = trainer.fit()
```

`prepare_model` handles device placement and the `DDP` wrapping for you.
`ScalingConfig` sets workers/GPUs; a shared `storage_path` holds checkpoints.

Feed training from a Ray Data pipeline

Add config + checkpoints, then stream sharded training batches from a Ray Data pipeline into each worker.

```
def train_loop_ray_data(config):
    model = ray.train.torch.prepare_model(
        TwoTower(config['num_users'],
                  config['num_items'], config['dim']))
    opt = torch.optim.Adam(model.parameters(), lr=1e-3)

    # this worker's slice of the stream:
    shard = get_dataset_shard("train")
    for batch in shard.iter_torch_batches(batch_size=1000):
        loss = step(model, batch)
        # rank-0 saves; report metrics + checkpoint
        if get_context().get_world_rank() == 0:
            torch.save(model.module.state_dict(), path)
            ckpt = Checkpoint.from_directory(d)
            ray.train.report({'loss': loss.item()},
                             checkpoint=ckpt)
```

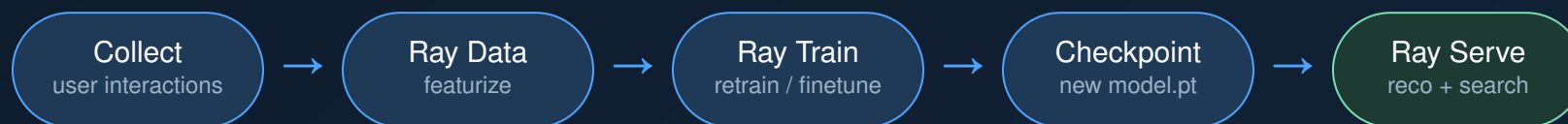
```
trainer = TorchTrainer(
    train_loop_ray_data,
    scaling_config=ScalingConfig(
        num_workers=2, use_gpu=True),
    train_loop_config={...},
    # supply the data pipeline:
    datasets={'train': training_pipeline},
)
result = trainer.fit()

# multi-epoch streams (re-reads) data each epoch.
# to reuse it, cache across the object store:
cached = training_pipeline.materialize()
```

Ray Data shards data per worker and prefetches batches; Ray schedules data-prep and training on different hardware. **materialize()** caches when re-streaming each epoch isn't worth it.

The continuous loop — train, serve, repeat

Wrap the training program in a Ray / Anyscale **Job**, scheduled to consume new interaction data and publish fresh checkpoints to a known location that Serve loads.



Takeaway: batch inference, online serving, and retraining are all the same toolkit — Ray Data pipelines, actors, and Serve deployments on one cluster — so a recommender can be served and continuously updated with minimal, composable code.